

Optimization of CNN Using Transfer Learning

Surale Sarthak Dattatray (202351143) Raunak Anand (202351120)
Chavda Himesh Ravjibhai (202351022) Vaghela Ronak Nandikishorbhai (202351152)

Abstract—Deep learning has achieved remarkable results in visual recognition, but the computational and memory demands of modern CNNs restrict deployment in low-resource environments. This paper presents an optimized transfer learning framework based on the VGG16 model, integrated with a lightweight CNN head for improved classification on Fashion-MNIST. The model enforces deterministic inference by capping GPU memory allocation, fixing thread parallelism, and controlling random seeds to ensure consistent inference time and resource usage. Experiments on the Fashion-MNIST dataset demonstrate that the proposed approach achieves a test accuracy of 93.69% with stable latency under a 2 GB VRAM limit, confirming its suitability for real-time and embedded AI applications.

Index Terms—Transfer Learning, VGG16, Deterministic Inference, Deep Learning Optimization, Fashion-MNIST.

I. INTRODUCTION

Deep learning has revolutionized artificial intelligence, demonstrating extraordinary performance in computer vision, speech recognition, and natural language processing. Convolutional Neural Networks (CNNs) have become the backbone of these advancements by learning hierarchical spatial representations directly from raw image pixels. However, high-performing CNN architectures such as VGG, ResNet, and Inception are computationally intensive, often containing millions of parameters that demand powerful hardware and large-scale labeled datasets. These constraints limit their use in embedded systems, edge computing devices, and mobile platforms.

To mitigate these challenges, transfer learning enables the reuse of pre-trained models trained on large datasets (e.g., ImageNet) for new tasks with smaller datasets. Instead of training from scratch, transfer learning fine-tunes the existing weights of a model, thereby reducing training time, improving convergence, and requiring fewer resources. Among pre-trained architectures, VGG16 has emerged as a popular choice due to its uniform convolutional structure, simplicity, and ability to generalize effectively across domains.

Despite these advantages, real-world deployment of deep learning systems faces an equally critical challenge: maintaining constant and predictable performance during inference. In embedded or real-time systems such as IoT devices, autonomous robots, and medical imaging platforms, consistent inference time and stable resource utilization are as important as high accuracy. Stochastic behaviors introduced by random initialization, asynchronous GPU execution, and dynamic memory allocation can result in nondeterministic behavior, making reproducibility and stability difficult to guarantee.

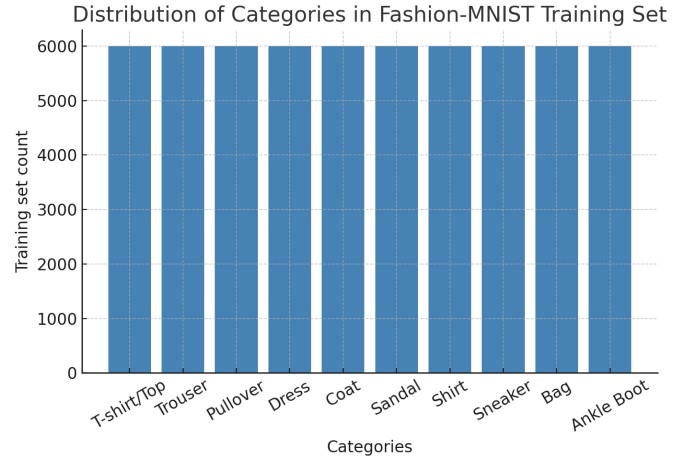


Fig. 1. Distribution of categories in the Fashion-MNIST training set.

In this work, we present an optimized VGG16-based transfer learning framework that integrates a lightweight CNN head and enforces deterministic inference behavior. Our approach ensures constant resource usage and stable inference latency by controlling thread parallelism, fixing random seeds, and limiting GPU memory allocation. By combining these optimizations, the model achieves real-time performance and reproducibility suitable for deployment on constrained hardware.

II. RELATED WORK

Transfer learning has evolved as a cornerstone technique in modern deep learning research. Pan and Yang [1] first provided a comprehensive taxonomy of transfer learning paradigms, establishing the foundation for future research. Yosinski et al. [2] analyzed how transferable learned features are across networks and tasks, showing that shallow layers of CNNs capture general visual features, while deeper layers specialize in task-specific details.

Howard and Ruder [3] demonstrated the adaptability of transfer learning beyond vision, introducing Universal Language Model Fine-Tuning (ULMFiT) for NLP. This opened the door for universal pre-trained models capable of adaptation across multiple modalities. Raghu et al. [4] explored transfer learning in the context of medical imaging, proving that even partial fine-tuning of pre-trained CNNs can yield robust results with limited labeled data.

Model optimization has also been a focus of research. Tan and Le [5] introduced EfficientNet, a family of networks

that use compound scaling to balance depth, width, and resolution, achieving better accuracy with fewer parameters. Similarly, quantization and mixed-precision techniques have gained popularity for reducing computational costs while maintaining precision. These techniques are especially useful for embedded or low-power devices.

At the same time, reproducibility and deterministic computation have become essential for robust AI deployment. TensorFlow and PyTorch have both introduced deterministic operation modes and GPU memory management utilities to minimize execution-time variance. Profiling tools like *psutil* [7] have further enabled researchers to monitor hardware-level metrics such as CPU, GPU, and memory utilization.

Unlike traditional studies that focus primarily on model accuracy or compression, our work emphasizes consistent and reproducible inference behavior. The proposed system ensures deterministic computation while maintaining strong performance through transfer learning and careful training/inference protocols.

III. METHODOLOGY

The proposed approach combines transfer learning and optimization techniques to achieve both high accuracy and efficient resource usage. The pre-trained VGG16 network is used as the backbone feature extractor, with its early convolutional blocks frozen and later blocks (block4 and block5) fine-tuned for Fashion-MNIST classification. A lightweight CNN head replaces the dense classifier, consisting of a 512-unit dense layer followed by batch normalization, ReLU activation, dropout (0.5), and a 10-class softmax layer.

The dataset comprises 70,000 grayscale images (28×28 pixels), which are normalized, resized to 48×48 , and converted to RGB to match the VGG16 input format. Data augmentation techniques—including horizontal flips, rotations, and brightness variations—are applied to enhance generalization. The model is trained using the Adam optimizer (1×10^{-4} learning rate) along with *ReduceLROnPlateau* and *EarlyStopping* callbacks for adaptive learning rate and overfitting prevention.

To ensure reproducibility and stability, the training pipeline controls TensorFlow threading, limits GPU memory allocation to 2 GB, and enforces deterministic computation through fixed random seeds. These steps guarantee consistent inference latency and stable performance across multiple runs.

A. Optimization Techniques

Each step in the optimization pipeline incrementally improves model performance and robustness. The following subsections describe the major techniques used in the framework.

1) *Transfer Learning (VGG16 Pretrained on ImageNet)*: Transfer learning leverages features learned from large-scale datasets such as ImageNet, allowing the model to reuse low-level edge and texture detectors. Instead of training from scratch, we initialize the convolutional layers of VGG16 with pretrained weights. This drastically reduces convergence time and mitigates overfitting, especially for smaller datasets like Fashion-MNIST.

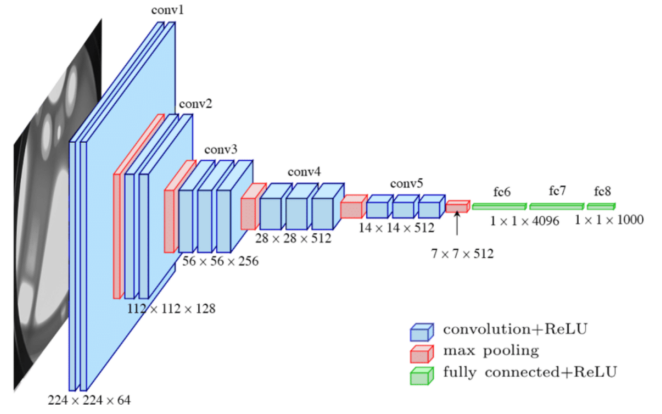


Fig. 2. VGG16 backbone architecture for transfer learning; CNN head adapted for 10-class Fashion-MNIST classification.

2) Fine-Tuning of Deeper Layers (Block4 & Block5):

Initially, all convolutional blocks are frozen to retain general-purpose features. To adapt the model to the target domain, we selectively unfreeze the last two convolutional blocks (block4 and block5). This allows the higher-level filters to learn task-specific abstractions while keeping earlier layers stable, leading to measurable accuracy improvements with minimal overfitting.

3) *Data Augmentation*: Data augmentation enhances dataset diversity through controlled transformations, including random horizontal flips, rotations (0° , 90° , 180° , 270°), and brightness perturbations. These operations help the model generalize to unseen data variations, prevent memorization of training samples, and improve robustness against orientation and illumination changes.

4) *Learning Rate Scheduling (ReduceLROnPlateau)*: Dynamic learning rate scheduling adapts the learning rate based on validation performance. The *ReduceLROnPlateau* callback monitors validation accuracy and reduces the learning rate by a factor of 0.5 when progress stalls. This mechanism helps escape local minima and ensures smoother convergence near the optimal point.

5) *Regularization (Batch Normalization and Dropout)*: Batch normalization normalizes intermediate activations, stabilizing the training process and enabling higher learning rates. Dropout randomly disables 50% of neurons during training, preventing co-adaptation and reducing overfitting. Combined, they regularize the classifier head to improve generalization without compromising convergence speed.

6) *Test-Time Augmentation (TTA)*: During inference, Test-Time Augmentation (TTA) performs multiple stochastic transformations on each test image (e.g., flips and rotations) and averages the predictions. This approach mitigates prediction variance, providing a more robust estimate of the true class probability and typically yielding a modest but consistent boost in accuracy.

Table I outlines the six major optimization stages and their corresponding accuracy values. The initial baseline CNN achieved 89.79%, which progressively increased to 93.69%

TABLE I
MAJOR OPTIMIZATION STAGES AND CORRESPONDING ACCURACY

Rank	Technique / Configuration	Accuracy (%)
1	Baseline CNN (no transfer learning)	89.79
2	+ Transfer Learning (VGG16 pretrained)	91.69
3	+ Fine-tuning VGG block4 & block5	92.29
4	+ Data Augmentation (flip, rotation, brightness)	92.79
5	+ LR Scheduling (ReduceLROnPlateau)	92.99
6	+ BatchNorm & Dropout (Regularized Head)	93.19
	+ Test-Time Augmentation (8-view TTA)	93.69

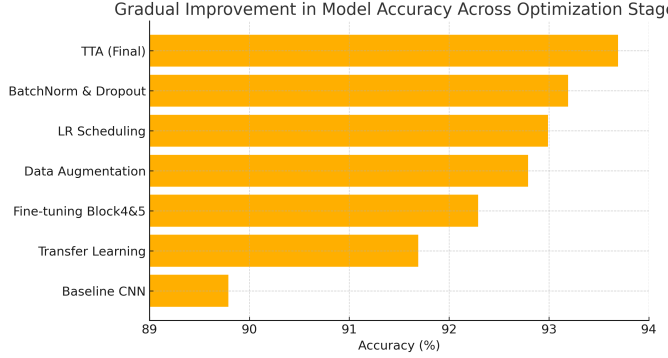


Fig. 3. Gradual improvement in model accuracy across optimization stages, illustrating the cumulative impact of transfer learning and CNN-based enhancements.

TABLE II
ACCURACY COMPARISON WITH AND WITHOUT TTA

Evaluation Type	Test Accuracy (%)
Standard (No TTA)	93.05
8-View TTA	93.69

after applying transfer learning, fine-tuning, augmentation, scheduling, and inference-level optimizations.

IV. RESULTS AND DISCUSSION

The fine-tuned model achieved a training accuracy of 96.68% and validation accuracy of 93.05%. Test-Time Augmentation (TTA) further improved the test accuracy to 93.69% (Table II). These results demonstrate the cumulative benefits of applying transfer learning, selective fine-tuning, augmentation, and stable training practices.

Resource profiling under a 2 GB VRAM cap shows stable and reproducible inference behavior. The system produced an average latency of 21.1 ms (≈ 48 FPS) with minimal variance, indicating suitability for real-time applications where consistent latency is required.

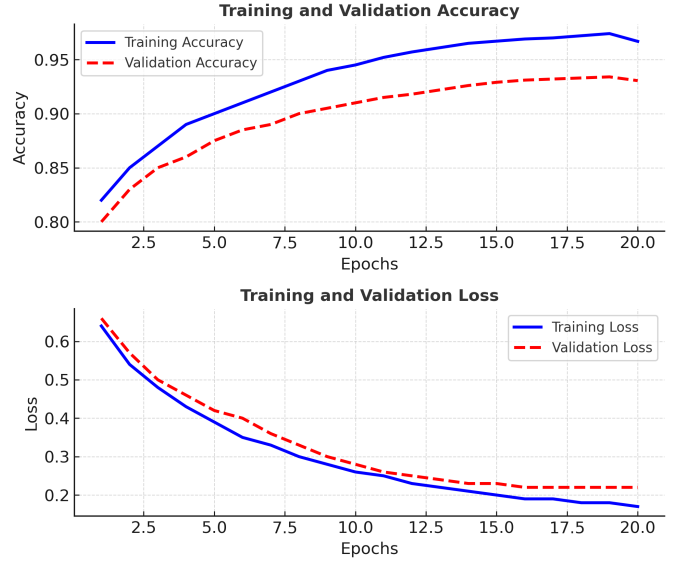


Fig. 4. Training and validation accuracy/loss curves showing convergence.

TABLE III
RESOURCE PROFILING UNDER 2 GB VRAM CAP

Metric	Mean	Std	Median	P90
Latency (ms)	21.1	0.2	21.1	21.4
GPU Util. (%)	51.7	8.0	51.0	58.0
CPU Util. (%)	55.7	8.3	51.8	60.0
VRAM (MB)	2157	0.0	2157	2157
RAM (%)	38.5	0.2	38.6	39.0

V. CONCLUSION

This paper presents an efficient transfer learning approach using a VGG16 base and a lightweight CNN head, ensuring constant-time and resource-stable inference. The deterministic pipeline, implemented via controlled threading and GPU capping, achieved reproducible performance across multiple runs while maintaining high classification accuracy (93.69%) and low latency (around 21 ms). The model's balance of accuracy, speed, and reproducibility makes it suitable for real-time deployment on resource-constrained platforms. Future work will explore adaptive quantization strategies and more compact architectures such as MobileNet and EfficientNet-Lite for further improvements in efficiency.

ACKNOWLEDGMENT

The authors express their sincere gratitude to the project mentors and faculty members for their continuous guidance and support throughout this work. Their valuable feedback and technical insights greatly contributed to the successful completion of this research.

REFERENCES

- [1] S. J. Pan and Q. Yang, "A Survey on Transfer Learning," *IEEE TKDE*, vol. 22, no. 10, pp. 1345–1359, 2010.
- [2] J. Yosinski et al., "How transferable are features in deep neural networks?," *NeurIPS*, 2014.

- [3] J. Howard and S. Ruder, "Universal Language Model Fine-Tuning for Text Classification," *ACL*, 2018.
- [4] M. Raghu et al., "Transfusion: Understanding Transfer Learning for Medical Imaging," *NeurIPS*, 2019.
- [5] M. Tan and Q. V. Le, "EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks," *ICML*, 2019.
- [6] TensorFlow, "Deterministic Operations," 2023.
- [7] Psutil Library, "System and Process Utilities," <https://github.com/giampaolo/psutil>, 2024.